Guía de CSS — Temario de Examen

Tabla de Contenidos

1. Modelo de Caja (Box Model)
2. Estilos de Borde
3. Etiquetas y propiedades clave
4. Selectores: padres, hijos y hermanos
5. Overflow y Float
6. Combinadores (Combinators)
7. Pseudoclases vs Pseudoelementos
8. La regla !important
9. Website Layout
10. Flexbox
11. Grid
12. Especificidad (Specificity)
13. Unidades: píxeles, porcentajes y valores relativos
14. Buenas prácticas de CSS en cascada

---

1. Modelo de Caja (Box Model)

Todo elemento HTML es una caja rectangular compuesta por cuatro capas:

```
+-----------------------------------------+
|                margin                   |
|  +-----------------------------------+  |
|  |             border                |  |
|  |  +-----------------------------+  |  |
|  |  |          padding            |  |  |
|  |  |  +-----------------------+  |  |  |
|  |  |  |        content        |  |  |  |
|  |  |  +-----------------------+  |  |  |
|  |  +-----------------------------+  |  |
|  +-----------------------------------+  |
+-----------------------------------------+
```

- **Content:** el contenido real (texto, imagen...)
- **Padding:** espacio entre el contenido y el borde. Forma parte visible del elemento.
- **Border:** línea que rodea el padding y el contenido.
- **Margin:** espacio exterior al borde. Separa el elemento de los demás. No tiene color visible.

```
div {
  width: 300px;
  padding: 20px;
  border: 5px solid black;
  margin: 10px;
}
/* Tamaño total visible = 300 + 20*2 + 5*2 = 350px */
```

Shorthand de margin y padding

```
/* 1 valor → todos los lados */
margin: 20px;

/* 2 valores → arriba/abajo   izquierda/derecha */
margin: 10px 20px;

/* 3 valores → arriba   izquierda/derecha   abajo */
margin: 10px 20px 30px;

/* 4 valores → arriba   derecha   abajo   izquierda (sentido horario) */
margin: 10px 20px 30px 40px;
```

Los mismos patrones funcionan para `padding`.

Por lado

```
margin-top: 10px;
margin-right: 20px;
margin-bottom: 10px;
margin-left: 20px;

padding-top: 10px;
padding-right: 20px;
padding-bottom: 10px;
padding-left: 20px;
```

Colapso de márgenes

Cuando dos márgenes verticales se tocan, solo se aplica el mayor de los dos (no se suman).

```
h1 { margin-bottom: 30px; }
p  { margin-top: 20px; }
/* Espacio entre ellos = 30px, no 50px */
```

box-sizing

```
/* content-box (default): width solo cuenta el content */
/* tamaño total = width + padding + border */
.caja-default {
  box-sizing: content-box;
  width: 200px;
  padding: 20px;
  border: 5px solid #333;
  /* visible = 200 + 40 + 10 = 250px */
}

/* border-box: width incluye padding y border */
/* tamaño total = width (incluye todo) */
.caja-border {
  box-sizing: border-box;
  width: 200px;
  padding: 20px;
  border: 5px solid #333;
  /* visible = exactamente 200px */
}

/* Práctica recomendada: aplicar globalmente */
*, *::before, *::after {
  box-sizing: border-box;
}
```

2. Estilos de Borde

border-style

```
p { border-style: dotted; }   /* punteado */
p { border-style: dashed; }   /* guiones */
p { border-style: solid; }    /* línea continua */
p { border-style: double; }   /* doble línea */
p { border-style: groove; }   /* efecto 3D hundido */
p { border-style: ridge; }    /* efecto 3D resaltado */
p { border-style: inset; }    /* borde insertado */
p { border-style: outset; }   /* borde saliente */
p { border-style: none; }     /* sin borde */
p { border-style: hidden; }   /* oculto */

/* Distintos estilos por lado: top right bottom left */
p { border-style: dotted dashed solid double; }
```

border-width y border-color

```
p {
  border-width: 2px;
  border-color: red;
}

/* Por lado */
p {
  border-width: 1px 2px 3px 4px;
  border-color: red green blue yellow;
}
```

Shorthand

```
/* border: grosor estilo color */
p { border: 2px solid #333; }

/* Por lado */
p {
  border-top: 3px dashed green;
  border-right: 2px solid blue;
  border-bottom: 4px dotted red;
  border-left: 1px solid gray;
}
```

Bordes redondeados

```
div { border-radius: 10px; }
div { border-radius: 50%; }  /* círculo */
div { border-radius: 10px 20px 30px 40px; }
/*                   top-left top-right bottom-right bottom-left */
```

---

3. Etiquetas y propiedades clave

Propiedades de fondo

```
div {
  background-color: #f0f0f0;
  background-image: url("fondo.png");
  background-repeat: no-repeat;   /* repeat | repeat-x | repeat-y | no-repeat */
  background-position: center top;
  background-size: cover;         /* cover | contain | 100px 200px */
  background-attachment: fixed;   /* scroll | fixed */

  /* Shorthand */
  background: #f0f0f0 url("fondo.png") no-repeat center/cover;
}
```

Propiedades de texto

```
p {
  color: #333;
  text-align: center;            /* left | right | center | justify */
  text-decoration: underline;    /* none | underline | overline | line-through */
  text-transform: uppercase;     /* none | uppercase | lowercase | capitalize */
  text-indent: 40px;
  letter-spacing: 2px;
  word-spacing: 8px;
  line-height: 1.6;
  white-space: nowrap;
}
```

Propiedades de fuente

```
p {
  font-family: "Helvetica Neue", Arial, sans-serif;
  font-size: 16px;
  font-weight: bold;             /* normal | bold | 100–900 */
  font-style: italic;            /* normal | italic | oblique */
  font-variant: small-caps;

  /* Shorthand: style variant weight size/line-height family */
  font: italic small-caps bold 16px/1.5 Arial, sans-serif;
}
```

Alto y ancho

```
div {
  width: 500px;
  height: 200px;
  max-width: 100%;
  min-width: 200px;
  max-height: 400px;
  min-height: 100px;
}
```

Display

```
div    { display: block; }         /* ocupa toda la línea */
span   { display: inline; }        /* solo el espacio del contenido, no acepta
width/height */
li     { display: inline-block; }  /* inline pero acepta width/height */
div    { display: none; }          /* oculto y no ocupa espacio */

/* Diferencia con visibility: hidden */
div { visibility: hidden; }        /* oculto pero sigue ocupando espacio */
```

---

4. Selectores: padres, hijos y hermanos

Tipos de selectores básicos

```
*           { }   /* universal: todos los elementos */
p           { }   /* elemento */
.clase      { }   /* clase */
#id         { }   /* id */
h1, h2, p   { }   /* grupo (separa con coma) */
```

Relaciones entre elementos

```
<div>                <!-- padre de ul -->
  <ul>               <!-- hijo de div, padre de li -->
    <li>A</li>       <!-- hijo de ul, hermano de li -->
    <li>B</li>       <!-- hermano de li anterior -->
    <li>
      <span>C</span><!-- descendiente de div, hijo de li -->
    </li>
  </ul>
</div>
```

```
/* Descendente (espacio): cualquier nivel de profundidad */
div p { color: blue; }       /* todos los <p> dentro de <div> */

/* Hijo directo (>): solo el nivel inmediato */
ul > li { color: red; }      /* solo <li> que son hijos directos de <ul> */

/* Hermano adyacente (+): el inmediato siguiente */
h1 + p { font-weight: bold; }/* primer <p> justo después de <h1> */

/* Hermanos generales (~): todos los siguientes */
h1 ~ p { color: gray; }      /* todos los <p> hermanos de <h1> */
```

Selectores de atributo

```
[title]           { }   /* tiene el atributo */
[type="text"]     { }   /* valor exacto */
[href^="https"]   { }   /* empieza con */
[href$=".pdf"]    { }   /* termina con */
[href*="google"]  { }   /* contiene en cualquier posición */
```

5. Overflow y Float

Overflow

```
div {
  overflow: visible;   /* default: el contenido se desborda visualmente */
  overflow: hidden;    /* recorta el contenido que se desborda */
  overflow: scroll;    /* siempre muestra barra de desplazamiento */
  overflow: auto;      /* barra de desplazamiento solo si es necesario */
}

/* Por eje */
div {
  overflow-x: hidden;
  overflow-y: auto;
}

/* Texto desbordado con puntos suspensivos */
p {
  overflow: hidden;
  white-space: nowrap;
  text-overflow: ellipsis;
  width: 200px;
}
```

Float

```
/* Flota el elemento a la izquierda o derecha */
/* El contenido circundante lo rodea */

img { float: left; }    /* texto rodea por la derecha */
img { float: right; }   /* texto rodea por la izquierda */
img { float: none; }    /* default */

/* Clear: impide que un elemento se coloque junto a un float */
div { clear: left; }    /* ningún float permitido a la izquierda */
div { clear: right; }
div { clear: both; }    /* ningún float en ninguno de los dos lados */

/* Clearfix: solución para contenedor que no envuelve a sus hijos flotados */
.clearfix::after {
  content: "";
  display: table;
  clear: both;
}
```

```html
<!-- Ejemplo float -->
<div class="clearfix">
  <img style="float: left; margin-right: 15px;" src="foto.jpg">
  <p>El texto rodea la imagen porque está flotada a la izquierda.</p>
</div>
```

---

6. Combinadores (Combinators)

Los combinadores permiten seleccionar elementos en función de su relación con otros. Hay **4** en CSS:

Combinador	Símbolo	Descripción
Descendente	(espacio)	Cualquier descendiente
Hijo directo	>	Solo hijos inmediatos
Hermano adyacente	+	El primer hermano siguiente
Hermanos generales	~	Todos los hermanos siguientes

```
/* 1. Descendente (espacio) */
/* Selecciona todos los <p> dentro de un <div>, sin importar el nivel */
div p {
  color: blue;
}

/* 2. Hijo directo (>) */
/* Solo <p> que sean hijos DIRECTOS de <div> */
div > p {
  color: red;
}

/* 3. Hermano adyacente (+) */
/* El PRIMER <p> que aparece inmediatamente después de <h1> */
h1 + p {
  font-size: 1.2em;
}

/* 4. Hermanos generales (~) */
/* TODOS los <p> que son hermanos de <h1> y van después de él */
h1 ~ p {
  color: gray;
}
```

```
<!-- Ejemplo para entender la diferencia > vs espacio -->
<div>
  <p>Hijo directo → afectado por "div > p"  Y  "div p"</p>
  <section>
    <p>Nieto → solo afectado por "div p"  (NO por "div > p")</p>
  </section>
</div>
```

7. Pseudoclases vs Pseudoelementos

Pseudoclases (:)

Seleccionan un elemento en un **estado** concreto o según su **posición** en el DOM.

```
/* Estados de interacción */
a:link    { color: blue; }       /* enlace no visitado */
a:visited { color: purple; }     /* enlace visitado */
a:hover   { color: red; }        /* al pasar el ratón */
a:active  { color: orange; }     /* durante el clic */
input:focus { outline: 2px solid blue; } /* elemento enfocado */
input:checked { }                /* checkbox/radio marcado */
input:disabled{ opacity: 0.5; } /* elemento deshabilitado */
input:valid   { border-color: green; }
input:invalid { border-color: red; }

/* Posición en el DOM */
li:first-child       { }   /* primer hijo */
li:last-child        { }   /* último hijo */
li:nth-child(2)      { }   /* segundo hijo exacto */
li:nth-child(odd)    { }   /* hijos impares (1, 3, 5…) */
li:nth-child(even)   { }   /* hijos pares (2, 4, 6…) */
p:first-of-type      { }   /* primer <p> de su tipo */
p:last-of-type       { }   /* último <p> de su tipo */

/* Negación */
p:not(.especial) { color: gray; }  /* <p> que NO tiene clase .especial */

/* Otros */
:root  { }  /* mayor especificidad que html */
:empty { }  /* elementos sin contenido */
```

Pseudoelementos (::)

Crean o seleccionan **una parte** de un elemento que no existe en el HTML.

```
/* ::before y ::after — insertan contenido */
h2::before {
  content: "👉 ";    /* la propiedad content es obligatoria */
}
p::after {
  content: " [leer más]";
  color: gray;
  font-size: 0.85em;
}

/* ::first-line — primera línea de texto */
p::first-line {
```

```
  font-weight: bold;
  color: #333;
}

/* ::first-letter — primera letra */
p::first-letter {
  font-size: 3em;
  float: left;
  margin-right: 5px;
  color: #c0392b;
}

/* ::selection — texto seleccionado por el usuario */
::selection {
  background-color: #ffeb3b;
  color: #333;
}

/* ::placeholder — placeholder de inputs */
input::placeholder {
  color: #aaa;
  font-style: italic;
}

/* ::marker — marcador de lista */
li::marker {
  color: coral;
}
```

Diferencia clave

	Pseudoclase	Pseudoelemento
Sintaxis	`:nombre`	`::nombre`
¿Qué hace?	Selecciona un elemento en un estado	Crea/selecciona una parte del elemento
Ejemplo	`a:hover`	`p::first-letter`
Existe en el HTML	Sí	No (es virtual)

> ⚠ **No mezclar propiedades**: `content` solo funciona en `::before` / `::after`. Las pseudoclases no tienen `content`.

---

8. La regla !important

```
/* !important anula cualquier otra declaración,
   independientemente de la especificidad */

p { color: blue !important; }

#parrafo { color: red; }
/* Normalmente el ID ganaría, pero !important se impone */
```

¿Para qué sirve?

Forzar que un estilo se aplique por encima de cualquier regla de la cascada. Útil en **clases de utilidad** que nunca deben sobreescribirse:

```
.oculto {
  display: none !important;
}

.texto-rojo {
  color: red !important;
}
```

⚠ Cuándo NO usarlo

Para sobreescribir un `!important` externo necesitas otro `!important` con igual o mayor especificidad, lo que genera código difícil de depurar. **Regla de oro:** úsalo solo como último recurso o en utilidades globales.

```
/* MAL: abuso de !important */
h1 { font-size: 2rem !important; }
p  { color: gray !important; }

/* BIEN: construir bien el selector */
.seccion-articulo h1 { font-size: 2rem; }
.seccion-articulo p  { color: gray; }
```

---

9. Website Layout

Un layout típico de sitio web se divide en estas secciones semánticas:

```
+---------------------------+
|          header           |
+---------------------------+
|            nav            |
+--------+------------------+
|        |                  |
| aside  |      main        |
|        |                  |
+--------+------------------+
|          footer           |
+---------------------------+
```

```html
<body>
  <header>Logo y título</header>
  <nav>Menú de navegación</nav>
  <div class="contenedor">
    <aside>Barra lateral</aside>
    <main>Contenido principal</main>
  </div>
  <footer>Pie de página</footer>
</body>
```

```css
/* Reset básico */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

/* Header */
header {
  background-color: #333;
  color: white;
  padding: 15px 20px;
  text-align: center;
}

/* Navegación */
nav {
  background-color: #444;
  overflow: hidden;
}
nav a {
```

```
  float: left;
  display: block;
  padding: 14px 20px;
  color: white;
  text-decoration: none;
}
nav a:hover {
  background-color: #666;
}

/* Contenedor de dos columnas */
.contenedor {
  display: flex;
}

/* Sidebar */
aside {
  width: 200px;
  background-color: #f4f4f4;
  padding: 20px;
  min-height: 400px;
}

/* Contenido principal */
main {
  flex: 1;
  padding: 20px;
}

/* Footer */
footer {
  background-color: #333;
  color: white;
  text-align: center;
  padding: 20px;
  clear: both;
}
```

---

10. Flexbox

Concepto

Flexbox es un modelo de layout **unidimensional**: trabaja en una fila O en una columna.

- **Flex container:** el elemento padre con `display: flex`.
- **Flex items:** los hijos directos del contenedor.

> ⚠ Las propiedades del contenedor se ponen en el padre. Las propiedades de los items se ponen en los hijos. **No mezclar.**

Propiedades del CONTENEDOR

```
.contenedor {
  display: flex;
  /* o display: inline-flex; */

  /* Dirección del eje principal */
  flex-direction: row;            /* → default */
  flex-direction: row-reverse;    /* ← */
  flex-direction: column;         /* ↓ */
  flex-direction: column-reverse; /* ↑ */

  /* ¿Los items se envuelven en nueva línea? */
  flex-wrap: nowrap;       /* default: todos en una línea */
  flex-wrap: wrap;         /* se envuelven hacia abajo */
  flex-wrap: wrap-reverse;

  /* Shorthand direction + wrap */
  flex-flow: row wrap;

  /* Alineación en el EJE PRINCIPAL */
  justify-content: flex-start;    /* default: al inicio */
  justify-content: flex-end;      /* al final */
  justify-content: center;        /* centrado */
  justify-content: space-between; /* espacio entre items (sin bordes) */
  justify-content: space-around;  /* espacio alrededor de cada item */
  justify-content: space-evenly;  /* espacio igual entre todo */

  /* Alineación en el EJE CRUZADO — una línea */
  align-items: stretch;     /* default: ocupan todo el alto */
  align-items: flex-start;  /* arriba */
  align-items: flex-end;    /* abajo */
  align-items: center;      /* centrado verticalmente */
  align-items: baseline;    /* alineado por la línea base del texto */

  /* Alineación en el EJE CRUZADO — múltiples líneas (solo con wrap) */
  align-content: flex-start;
  align-content: flex-end;
```

```
  align-content: center;
  align-content: space-between;
  align-content: space-around;
  align-content: stretch;   /* default */

  /* Espacio entre items */
  gap: 20px;
  gap: 10px 20px;   /* fila columna */
  row-gap: 10px;
  column-gap: 20px;
}
```

Propiedades de los ITEMS

```
.item {
  /* Cuánto crece para ocupar espacio libre */
  flex-grow: 0;      /* default: no crece */
  flex-grow: 1;      /* crece proporcionalmente */

  /* Cuánto se encoge si no hay espacio */
  flex-shrink: 1;    /* default: puede encogerse */
  flex-shrink: 0;    /* no se encoge */

  /* Tamaño base antes de crecer/encoger */
  flex-basis: auto;  /* default */
  flex-basis: 200px;

  /* Shorthand: grow shrink basis */
  flex: 0 1 auto;    /* default */
  flex: 1;           /* equivale a flex: 1 1 0 (crece, se encoge, base 0) */
  flex: 0 0 200px;   /* tamaño fijo */

  /* Orden de aparición (default 0) */
  order: -1;   /* aparece antes */
  order: 2;    /* aparece después */

  /* Alineación individual (sobreescribe align-items del padre) */
  align-self: auto;
  align-self: flex-start;
  align-self: flex-end;
  align-self: center;
  align-self: stretch;
}
```

Ejemplos prácticos

```
/* Centrar perfectamente */
.centrar {
```

```
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

/* Navbar */
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0 20px;
}

/* Columna principal + sidebar */
.layout {
  display: flex;
  gap: 20px;
}
.sidebar  { flex: 0 0 250px; } /* fijo en 250px */
.main     { flex: 1; }         /* ocupa el resto */
```

---

11. Grid

Concepto

Grid es un modelo de layout **bidimensional**: trabaja con filas Y columnas a la vez.

- **Grid container:** el elemento padre con `display: grid`.
- **Grid items:** los hijos directos del contenedor.

Diferencias con Flexbox

	Flexbox	Grid
Dimensiones	1D (fila o columna)	2D (filas y columnas)
Ideal para	Componentes, barras de nav	Layouts de página completa
Colocación	Flujo automático	Control exacto de posición

Propiedades del CONTENEDOR

```
.contenedor {
  display: grid;

  /* Definir columnas */
  grid-template-columns: 200px 1fr 200px;
  grid-template-columns: repeat(3, 1fr);          /* 3 columnas iguales */

  /* Definir filas */
  grid-template-rows: 100px auto 60px;
  grid-template-rows: repeat(2, 200px);

  /* Áreas con nombre */
  grid-template-areas:
    "header  header  header"
    "sidebar main    main"
    "footer  footer  footer";

  /* Espacio entre celdas */
  gap: 20px;
  gap: 10px 20px;  /* fila columna */
  row-gap: 10px;
  column-gap: 20px;

  /* Alineación de items dentro de su celda */
  justify-items: stretch;   /* horizontal: start | end | center | stretch */
  align-items: stretch;     /* vertical:   start | end | center | stretch */

  /* Alineación del grid completo dentro del contenedor */
```

```
  justify-content: start;
  align-content: start;

  /* Tamaño automático de filas implícitas */
  grid-auto-rows: minmax(100px, auto);

  /* Dirección de flujo automático */
  grid-auto-flow: row;      /* row(default) | column | dense */
}
```

Propiedades de los ITEMS

```
.item {
  /* Posicionamiento por número de línea */
  grid-column: 1 / 3;        /* desde línea 1 hasta línea 3 */
  grid-column: 1 / span 2;   /* empieza en 1, abarca 2 columnas */
  grid-column: 1 / -1;       /* desde la primera hasta la última línea */

  grid-row: 1 / 2;
  grid-row: 2 / span 3;

  /* Asignar a un área nombrada */
  grid-area: header;
  grid-area: sidebar;
  grid-area: main;
  grid-area: footer;

  /* Alineación individual */
  justify-self: center;      /* horizontal dentro de su celda */
  align-self: center;        /* vertical dentro de su celda */
}
```

Ejemplo: layout de página con áreas

```
.pagina {
  display: grid;
  grid-template-areas:
    "header  header"
    "sidebar main"
    "footer  footer";
  grid-template-columns: 220px 1fr;
  grid-template-rows: auto 1fr auto;
  min-height: 100vh;
  gap: 10px;
}

header   { grid-area: header; }
.sidebar { grid-area: sidebar; }
```

```
main     { grid-area: main; }
footer   { grid-area: footer; }
```

La unidad `fr` y `minmax()`

```
/* fr = fracción del espacio disponible */
grid-template-columns: 1fr 2fr 1fr;
/* la del medio es el doble de ancha que las laterales */

/* minmax(): tamaño mínimo y máximo */
grid-template-columns: repeat(3, minmax(150px, 1fr));
```

---

12. Especificidad (Specificity)

La especificidad determina qué regla CSS "gana" cuando varias reglas afectan al mismo elemento.

Jerarquía (de mayor a menor)

Nivel	Tipo	Puntos
1	`style=""` (inline)	1000
2	`#id`	100
3	`.clase`, `[atributo]`, `:pseudoclase`	10
4	`elemento`, `::pseudoelemento`	1
5	`*` selector universal	0
+	`!important`	Anula todo

Cómo calcular

```
p              { }   /* (0,0,0,1) →   1 punto */
.nav           { }   /* (0,0,1,0) →  10 puntos */
#menu          { }   /* (0,1,0,0) → 100 puntos */
div.nav p      { }   /* (0,0,1,2) →  12 puntos */
#menu .item a  { }   /* (0,1,1,1) → 111 puntos */
```

Regla de desempate

Si dos reglas tienen **la misma especificidad**, gana la que aparece **última** en el CSS.

```
p { color: red; }
p { color: blue; }  /* ← gana esta (está después) */
```

Por qué importa construir buenos selectores

```
/* MAL: selector innecesariamente específico → difícil de sobreescribir */
body div#contenido ul.lista li a { color: red; }

/* BIEN: simple y suficiente */
.lista a { color: red; }
```

Un selector simple es más fácil de sobreescribir, de mantener y de leer. Cuanto más específico sea un selector, más difícil será gestionarlo en CSS grandes.

13. Unidades: píxeles, porcentajes y valores relativos

Unidades absolutas

```
div { width: 300px; }    /* píxeles: fijo, independiente del contexto */
div { width: 2cm; }      /* centímetros */
p   { font-size: 12pt; } /* puntos (tipografía impresa) */
```

Unidades relativas

```
/* em: relativo al font-size del PADRE */
p { font-size: 1.5em; }
/* Si el padre tiene 16px → 1.5em = 24px */
/* Puede acumularse en elementos anidados */

/* rem: relativo al font-size del ELEMENTO RAÍZ (<html>) */
p { font-size: 1.5rem; }
/* Siempre 1.5 × font-size de <html>, más predecible que em */

/* %: relativo al padre */
div { width: 80%; }      /* 80% del ancho del padre */

/* vw / vh: porcentaje del VIEWPORT */
section { width: 50vw; }    /* 50% del ancho de la ventana */
section { height: 100vh; }  /* 100% del alto de la ventana */

/* vmin / vmax */
div { font-size: 5vmin; }   /* 5% del lado más pequeño del viewport */

/* ch: basado en el ancho del carácter "0" */
p { max-width: 65ch; }      /* línea de texto legible */
```

¿Cuándo usar cada una?

Situación	Unidad recomendada
Tamaños de fuente accesibles	`rem`
Espaciado interno de componentes	`em` o `px`
Ancho de contenedores	`%` o `vw`
Altura a pantalla completa	`vh`
Tamaños fijos e independientes	`px`
Líneas de texto legibles	`ch`

> No siempre se trabaja con píxeles. Los valores relativos como `%`, `em`, `rem` y `vw/vh` son esenciales para hacer diseños flexibles y accesibles.

---

14. Buenas prácticas de CSS en cascada

1. Selectores simples y con la mínima especificidad necesaria

```
/* MAL */
body div.contenedor ul#lista li.activo a { color: red; }

/* BIEN */
.lista .activo a { color: red; }
```

2. Evitar !important salvo en utilidades

```
/* Solo en casos excepcionales */
.sr-only {
  position: absolute !important;
  width: 1px !important;
  overflow: hidden !important;
}
```

3. Aprovechar la cascada: de lo general a lo específico

```
/* 1. Estilos base */
a { color: #007bff; text-decoration: none; }

/* 2. Estilos contextuales */
.nav a { color: white; }

/* 3. Estados */
.nav a:hover { text-decoration: underline; }
```

4. Usar variables CSS para coherencia

```
:root {
  --color-primario: #007bff;
  --espacio: 16px;
  --radio: 8px;
}

.btn {
  background: var(--color-primario);
  padding: var(--espacio);
  border-radius: var(--radio);
}
```

5. Preferir unidades relativas

```
/* MAL: tamaños fijos que no escalan */
p { font-size: 14px; line-height: 20px; }

/* BIEN: escalable y accesible */
p { font-size: 0.875rem; line-height: 1.5; }
```

6. box-sizing global

```
*, *::before, *::after {
  box-sizing: border-box;
}
```

7. No repetir propiedades: usar selectores de grupo

```
/* MAL */
h1 { font-family: Arial; color: #333; }
h2 { font-family: Arial; color: #333; }
h3 { font-family: Arial; color: #333; }

/* BIEN */
h1, h2, h3 {
  font-family: Arial;
  color: #333;
}
```

8. Accesibilidad: nunca eliminar el focus

```
/* MAL */
:focus { outline: none; }

/* BIEN: personalizar, no eliminar */
:focus-visible {
  outline: 3px solid #005fcc;
  outline-offset: 2px;
}
```